

Modificaciones y mejoras al diseño del sistema de alquiler “Los Atuendos”

Caso: negocio de alquiler “Los atuendos”

Patrones de diseño de software

Estudiantes: Jose Manuel Castro López

Sergio Alejandro Parra Arango

Grupo: 2310-8B MOM 1 VIRTUAL

Ingeniería de Software

Marzo de 2025

Contenido

1. Introducción.....	2
2. Estado inicial del sistema	2
3. Principales modificaciones realizadas.....	3
4. Modificaciones en patrones estructurales y creacionales	3
5. Modificaciones en patrones de comportamiento	5
6. Modificaciones en diagramas.....	6
7. Impacto de las mejoras	7
8. Conclusión.....	7

1. Introducción

El presente documento tiene como propósito describir y justificar las modificaciones realizadas al diseño del sistema de alquiler “Los Atuendos”, en comparación con su versión inicial. Estas mejoras se enfocaron en la incorporación de patrones de diseño de software, con el fin de optimizar la arquitectura del sistema, mejorar la mantenibilidad, escalabilidad y facilitar la evolución futura del aplicativo.

2. Estado inicial del sistema

En la versión inicial del proyecto, el sistema contaba con una arquitectura básica orientada a operaciones CRUD, donde:

- La lógica de negocio estaba centralizada en pocas clases.
- Existía un alto acoplamiento entre componentes.
- No se utilizaban patrones de diseño.

- Las funcionalidades eran limitadas (principalmente gestión de clientes).

Este enfoque dificultaba la extensión del sistema, el mantenimiento y la reutilización de componentes.

3. Principales modificaciones realizadas

3.1 Reestructuración de la arquitectura

Se realizó una reorganización completa del backend, separando responsabilidades en capas:

- Controladores (Controller)
- Servicios (Service)
- Repositorios (Repository)
- Paquete de patrones (patterns)

Justificación:

Esta separación permite aplicar el principio de responsabilidad única (SRP), facilitando el mantenimiento, pruebas y escalabilidad del sistema.

3.2 Implementación de patrones de diseño

Se incorporaron patrones de diseño clasificados en tres categorías: creacionales, estructurales y de comportamiento.

4. Modificaciones en patrones estructurales y creacionales

4.1 Factory Method

Cambio realizado: Se implementaron fábricas para la creación de objetos como prendas y alquileres.

Antes: Los objetos se instanciaban directamente.

Ahora: La creación se delega a clases especializadas (Factory).

Justificación: Permite desacoplar la creación de objetos y facilita la extensión del sistema al agregar nuevos tipos de prendas sin modificar código existente.

4.2 Singleton

Cambio realizado: Se implementó una clase Singleton para el control de instancias globales (ej. contador de alquileres).

Justificación: Garantiza un único punto de acceso y control, evitando inconsistencias en la información.

4.3 Composite

Cambio realizado: Se implementó una estructura jerárquica para manejar prendas individuales y agrupadas.

Justificación: Permite tratar de forma uniforme objetos simples y compuestos, facilitando la gestión de inventario.

4.4 Adapter

Cambio realizado: Se integró un adaptador para comunicación con sistemas externos (ej. lavandería).

Justificación: Permite compatibilidad entre interfaces diferentes sin modificar el sistema principal.

5 Decorator

Cambio realizado: Se implementó un decorador para añadir funcionalidades como logging en alquileres.

Justificación: Permite extender funcionalidades dinámicamente sin modificar clases base.

5. Modificaciones en patrones de comportamiento

5.1 Observer

Cambio realizado: Se implementó un sistema de notificaciones para cambios de estado de prendas.

Modificación del diseño: Se agregaron observadores (clientes y empleados) y un gestor de eventos.

Justificación: Desacopla la lógica de notificación, permitiendo agregar nuevos observadores sin afectar el sistema.

5.2 Strategy

Cambio realizado: Se implementaron diferentes estrategias de pago (efectivo, tarjeta, transferencia).

Modificación del diseño: Se definió una interfaz común para métodos de pago.

Justificación: Permite cambiar dinámicamente el algoritmo de pago sin modificar la lógica principal.

5.3 Command

Cambio realizado: Se encapsularon acciones como crear, cancelar y finalizar alquiler.

Modificación del diseño: Se introdujo una capa de comandos y un gestor de ejecución.

Justificación: Permite desacoplar la invocación de la ejecución, facilitando auditoría, historial y posibles funciones de deshacer.

5.4 Iterator

Cambio realizado: Se implementó un iterador para recorrer colecciones de prendas.

Justificación: Permite acceder a los elementos sin exponer la estructura interna de las colecciones.

6. Modificaciones en diagramas

6.1 Diagrama de clases

Cambios realizados:

- Inclusión de nuevas clases para cada patrón.
- Separación de responsabilidades.
- Incorporación de interfaces y jerarquías.

Motivo: Representar una arquitectura más modular, desacoplada y alineada con buenas prácticas de diseño.

6.2 Diagramas de secuencia

Cambios realizados:

Se agregaron nuevos diagramas para:

- Command (flujo de alquiler)
- Observer (cambio de estado de prenda)
- Strategy (proceso de pago)
- Iterator (recorrido de inventario)

Motivo: Visualizar claramente la interacción entre objetos y validar el comportamiento dinámico del sistema.

7. Impacto de las mejoras

Las modificaciones realizadas generaron los siguientes beneficios:

- Reducción del acoplamiento
- Mayor claridad en la estructura del sistema
- Facilidad para agregar nuevas funcionalidades
- Mejor organización del código
- Mayor alineación con principios SOLID

8. Conclusión

La evolución del sistema desde una arquitectura básica hacia una basada en patrones de diseño permitió transformar una solución simple en un sistema robusto, flexible y escalable. La implementación de patrones estructurales, creacionales y de comportamiento no solo mejoró la calidad del software, sino que también facilitó la comprensión del sistema y su futura expansión.

Estas mejoras evidencian la importancia del uso de patrones de diseño como herramienta fundamental en el desarrollo de software profesional.